

The bicriteria minimum spanning tree problem *

Yen Hung Chen

Department of Computer Science

Taipei Municipal University of Education, No.1, Taipei 10048, Taiwan, R.O.C

{yhchen}@tmue.edu.tw

Abstract

Let $G = (V, E)$ be an undirected graph with a weight function and a cost function on edges. The bicriteria minimum spanning tree problem is concerned with the determination of a minimum cost spanning tree T in G subject to the constraint that the total weight in T is at most a given bound B . In this paper, we present two polynomial time approximation schemes (PTASs) for the bicriteria minimum spanning tree problem.

1 Introduction

Motivated by the applications of quality of service (QoS) routing and multicasting, several multiple criteria problems have been studied [5, 9, 13, 14, 17, 19]. Marathe et al. [17] studied a class of *bicriteria network design* problems that are defined as follows. Given an undirected graph $G = (V, E)$ and two independent minimization criteria with a bound on the first criterion. A *generic bicriteria network design* problem involves the minimization of the second criterion but satisfies the bound on the first criterion among all possible subgraphs from G [17]. They considered three different criteria that have total edge-cost, diameter, and maximum degree of a graph. The total edge-cost of a graph is the sum of the costs of all edges in the subgraph. The diameter of a graph is the maximum distance between any pair of nodes in the subgraph. The degree of a node is the number of nodes adjacent to this node and the maximum degree of a graph is maximum over the degrees of all nodes in the subgraph. Let $G = (V, E)$ be an undirected graph and a positive integer B . Two different nonnegative functions, *weight* and *cost* associate on edges in E ,

respectively. The *bicriteria minimum spanning tree* (BMST) (also called as constrained minimum spanning tree) problem [1, 11, 12, 18] is concerned with the determination of a minimum total cost spanning tree T in G subject to the constraint that the total weight in T is at most B . Aggarwal et al. [1] showed that the BMST problem is weakly NP-hard. Then Hong, Chung, and Park [12] gave a pseudo-polynomial time algorithm to solve the BMST problem. Moreover, some approximation algorithms had been proposed to solve the BMST problem [11, 17, 18].

Given two positive real numbers α and β , An (α, β) -approximation algorithm for the BMST problem is defined as a polynomial-time algorithm that produces a solution with the total weight at most α times the bound B , and the total cost at most β times the total cost of optimal solution for the BMST problem. The approximation ratio is denoted by (α, β) . An (α, β) -approximation algorithm for the BMST problem is called as the polynomial time approximation scheme (PTAS) that produces a spanning tree with an approximation ratio of $(1, 1 + \varepsilon)$, for any constant $\varepsilon > 0$. For any constant $\varepsilon > 0$, Marathe et al. [17] gave a $(1 + \varepsilon, 1 + \frac{1}{\varepsilon})$ -approximation algorithm for the BMST problem. Ravi and Goemans [18] presented a $(2, 1)$ -approximation algorithm to solve the BMST problem in $O(|E| \log^2 |V| + |V| \log^3 |V|)$ time and improved the ratio to $(1 + \varepsilon, 1)$, where the time complexity is $O(|V|^{O(\frac{1}{\varepsilon})} (|E| \log^2 |V| + |V| \log^3 |V|))$ for any constant $\varepsilon > 0$. These algorithms are based on Lagrangian relaxation. Hassin and Levin [11] employed Ravi and Goemans' [18] algorithm with the matroid intersection strategy [2, 6, 15] to design a $(1, 1 + 5\varepsilon)$ -approximation algorithm in $O((\frac{1}{\varepsilon^2})^{(\frac{1}{\varepsilon})} |V|^3)$ time for the BMST problem. For other related bicriteria problems, Hassin [10] gave a fully polynomial time approximation scheme (FPTAS) for the *restricted shortest path* problem. The purpose of this problem is to find a minimum cost shortest path P from a source to

*This work was partly supported by the National Science Council of the Republic of China under grants NSC99-2221-E-133-003.

a destination subject to the constraint that the total weight in P is at most B . Then Lorenz and Raz [16] gave an improved FPTAS whose time complexity is $O(|V||E|\log\log|V| + \frac{|V||E|}{\epsilon})$ for the restricted shortest path problem. Further, Xue, Zhang, Tang, and Thulasiraman [20] also designed a FPTAS with improved time complexity to $O(|V||E|\log\log\log|V| + \frac{|V||E|}{\epsilon})$ for the restricted shortest path problem. If the graph is directed acyclic, Ergun, Sinha and Zhang [8] provided an improved FPTAS for the restricted shortest path problem with running time $O(\frac{|V||E|}{\epsilon})$. Chen and Xue [3, 4] used a similar method to design PTASs for the k -pair delay constrained minimum cost routing problem and the weight constrained Steiner tree problem in series-parallel graphs. These algorithms are based on the rounding and scaling strategy that is offered by Hassin [10]. In this paper, we present two polynomial time approximation schemes to find $(1, 1+\epsilon)$ -approximation ratio for the BMST problem. Applying the second PTAS to Hassin and Levin's algorithm [11], the approximation ratio can be improved to $(1, 1+4\epsilon)$ for the BMST problem.

The rest of this paper is organized as follows. In Section 2, we first describe a $(1, (1+\epsilon)^2)$ -approximation algorithm for the BMST problem, and then improve the approximation ratio to $(1, 1+\epsilon)$. Finally, we make a conclusion in Section 3.

2 PTASs for the Bicriteria Minimum Spanning Tree Problem

In this section, we first clarify a $(1, (1+\epsilon)^2)$ -approximation algorithm for the BMST problem. Hassin and Levin [11] first used this algorithm to obtain a $(1, 2(1+\epsilon))$ ratio for the BMST problem. Then they applied Lagrangian relaxation and matroid intersection methods to improve the ratio to $(1, 1+5\epsilon)$ in $O((\frac{1}{\epsilon^2})^{(\frac{1}{\epsilon})}|V|^3)$ time. However, they didn't describe the details of this algorithm. Then we improve this approximation ratio into $(1, 1+\epsilon)$. Note that applying our algorithm, the approximation ratio of Hassin and Levin's algorithm [11] can be improved to $(1, 1+4\epsilon)$ ratio with the same time complexity. For convenience, we use $C(H)$ (respectively, $W(H)$) to denote the sum of the costs (respectively, weights) of all edges in any subgraph H of G . Let T_{OPT} be the optimal solution of the BMST problem. For any constant $\epsilon > 0$ and an integer B , Ravi and Goemans' [18] algorithm, denoted by $\mathcal{A}_R(\epsilon, B)$,

produces a spanning tree T of the $W(T)$ at most $(1+\epsilon)B$, and of the $C(T)$ at most $C(T_{OPT})$. Our algorithm first finds a cost of the lower bound, denoted by LB , and a cost of the upper bound, denoted by UB , for the BMST problem. Then iteratively shrink this range between LB and UB until $UB \leq (1+\epsilon)LB$. A trivial lower bound LB can be set to 1 and an upper bound UB can be set to $|V| * C_{max}$, in which C_{max} is the maximum edge cost in G . However, the range between the trivial lower bound and upper bound is too large (i.e., not polynomial in input size). We reduce the range by Lorenz and Raz's method [16] such that $UB \leq |V| * LB$. First, let $c_1, c_2, \dots, c_k$ be all the distinct costs of edges in G . $G_j = (V, E_j)$ denotes the subgraph of $G(V, E)$, where E_j is the set of edges with costs not greater than c_j . It is clear that $G_k = G$ and $G_j \subseteq G_{j+1}$ for $1 \leq j \leq k-1$. For $1 \leq j \leq k$, we can use Prim's algorithm [7] to find a minimum weight spanning tree T_j in G_j . Let J be the smallest index j such that $W(T_j)$ is no more than B . Then finding the value c_J of a graph G can be done in $O(|V|^2 \log|V|)$ time by binary search and Prim's algorithm [7]. Therefore, we have $c_J \leq C(T_{OPT}) \leq |V| * c_J$. Hence, we can let the initial UB be $|V| * c_J$ and LB be c_J . If $UB \leq (1+\epsilon)LB$, then $C(T_J) \leq UB \leq (1+\epsilon)LB \leq (1+\epsilon)C(T_{OPT})$. Otherwise, we first swap the cost (respectively, weight) to the weight (respectively, cost) for each edge in G and find a value M between LB and UB . Then apply algorithm $\mathcal{A}_R(\epsilon, M)$ to find a spanning tree T in G . If the total weight of T is no more than B , let $UB = (1+\epsilon)M$; otherwise, let $LB = M$. Hence, the range between LB and UB is shrunk. Then we repeatedly run the algorithm $\mathcal{A}_R(\epsilon, M)$ until $UB \leq (1+\epsilon)LB$. Next lemma shows that we can obtain either a lower bound M or an upper bound $(1+\epsilon)M$ for the BMST problem after running $\mathcal{A}_R(\epsilon, M)$.

Lemma 1: Given an instance of the BMST problem with a constant $\epsilon > 0$ and an integer $\mathcal{C}$. We swap the cost (respectively, weight) to the weight (respectively, cost) for each edge, and then apply algorithm $\mathcal{A}_R(\epsilon, \mathcal{C})$ to find a spanning tree T . If the $W(T) \leq B$, then there is an upper bound $(1+\epsilon)\mathcal{C}$. Otherwise, we can obtain a lower bound $\mathcal{C}$.

Proof: For convenience, we use $T_{\mathcal{C}}$ to denote the minimum weight spanning tree of G with total cost at most $\mathcal{C}$. After running the algorithm $\mathcal{A}_R(\epsilon, \mathcal{C})$, we have a spanning tree T with $W(T) \leq W(T_{\mathcal{C}})$ and $C(T) \leq (1+\epsilon)\mathcal{C}$ since the costs and

weights of all edges are exchanged. Hence, if $W(T) \leq B$, then $C(T_{OPT}) \leq C(T) \leq (1 + \varepsilon)C$. Otherwise, $W(T_C) \geq W(T) > B$. Then we assume $C(T_{OPT}) < C$ for contradiction. Hence, we have $W(T_C) \leq W(T_{OPT})$. However, $W(T_{OPT}) \leq B$, we have $W(T_C) \leq B$. Hence, C must be less than or equal to $C(T_{OPT})$. $\square$

For clarification, we describe the $(1, (1 + \varepsilon)^2)$ -approximation algorithm for the BMST problem as follows.

Algorithm PTAS-BMST:

Input: A graph $G = (V, E)$ with a weight function and a cost function on edges, an integer bound B , and two real numbers $\varepsilon > 0$ and $\rho > 0$.

Output: A spanning tree T_{APX} of G subject to $W(T_{APX}) \leq B$.

1. **For** each edge in E , swap cost to weight and weight to cost, respectively. **end for**
2. Let an initial upper bound $UB = |V| * c_J$ and an initial lower bound $LB = c_J$.
3. **If** $UB \leq (1 + \varepsilon)LB$ **then** let $T_{APX} = T_J$ and **return**.

Else

- 3.1. Let $L = LB$, and $U = \lceil \frac{UB}{1+\rho} \rceil$.
- 3.2. Repeat the following steps until the condition (1) $U \leq (1 + \varepsilon)L$ or (2) $U \leq L + 1$ holds.
 - 3.2.1. Let a middle value $M = \sqrt{U * L}$.
 - 3.2.2. Use algorithm $\mathcal{A}_R(\rho, M)$ to find a spanning tree T .
 - 3.2.3. **/* if** $W(T) > B$ **then we have a lower bound** M **else we have an upper bound** $(1 + \rho)M$ **and** $C(T)$ **is less than or equal to this upper bound.** ***/**
If $W(T) \leq B$ **then** let $U = M$ and $T_{APX} = T$ **else** let $L = M$.

The approximation ratio of Algorithm PTAS-BMST is shown in the next lemma.

Lemma 2: Algorithm PTAS-BMST returns a spanning tree T_{APX} with $C(T_{APX}) \leq (1 + \varepsilon)(1 + \rho)C(T_{OPT})$.

Proof: Let the upper bound U_i , the middle value M_i , and the lower bound L_i be parameters at the beginning of the i -th iteration of step 3.2 for Algorithm PTAS-BMST. Let T_i denote a spanning tree after running the i -th iteration of step 3.2.2. It is clear $L_i \leq M_i \leq U_i$. Hence, $U_{i+1} \leq U_i$ and $L_{i+1} \geq L_i$. Moreover, we have $L_{i+1} = \sqrt{U_i * L_i}$ and $U_{i+1} = U_i$ when $W(T_i) > B$. Then we also have $U_{i+1} = \sqrt{U_i * L_i}$ and $L_{i+1} = L_i$ when $W(T_i) \leq B$. Further, $\frac{U_{i+1}}{L_{i+1}} = \frac{U_i}{\sqrt{U_i * L_i}}$ or $\frac{U_{i+1}}{L_{i+1}} = \frac{\sqrt{U_i * L_i}}{L_i}$. Therefore, we have $\frac{U_{i+1}}{L_{i+1}} = \sqrt{\frac{U_i}{L_i}}$. Note that it successively narrows down the range between the lower bound L and the upper bound U after each iteration of step 3.2. After running the i -th iteration of step 3.2.3, we obtain either a lower bound M_i or an upper bound $(1 + \rho)M_i$ by Lemma 1. Let k be the number of iterations of step 3.2. After performing this algorithm, we have either $U_{k+1} \leq (1 + \varepsilon)L_{k+1}$ or $U_{k+1} \leq L_{k+1} + 1$. For the former case, we have

$$\begin{aligned} C(T_{APX}) &\leq (1 + \rho)U_{k+1} \leq (1 + \varepsilon)(1 + \rho)L_{k+1} \\ &\leq (1 + \varepsilon)(1 + \rho)C(T_{OPT}). \end{aligned}$$

For the latter case, we have

$$\begin{aligned} C(T_{APX}) &\leq (1 + \rho)U_{k+1} \leq (1 + \rho)(L_{k+1} + 1) \\ &\leq (1 + \rho)(C(T_{OPT}) + 1). \end{aligned}$$

Let ρ be ε , and hence Algorithm PTAS-BMST achieves an approximation ratio of $(1, (1 + \varepsilon)^2)$ for the BMST problem. $\square$

Therefore, we have the following theorem.

Theorem 3: The BMST problem admits a PTAS. For any constant $\varepsilon > 0$, a $(1, (1 + \varepsilon)^2)$ -approximation algorithm for the BMST problem can be found in polynomial time.

Proof: By Lemma 2, Algorithm PTAS-BMST returns a $(1, (1 + \varepsilon)^2)$ -approximation solution for the BMST problem. Next, we analyze the time complexity of Algorithm PTAS-BMST as follows. Let k be the number of iterations of step 3.2. We also let L_1 and U_1 be the initial lower bound and upper bound, respectively. Since $\frac{U_{i+1}}{L_{i+1}} = \sqrt{\frac{U_i}{L_i}}$, we have

$$\begin{aligned} \frac{U_1}{L_1}, \\ \left(\frac{U_2}{L_2}\right) &= \left(\frac{U_1}{L_1}\right)^{\frac{1}{2}}, \\ \left(\frac{U_3}{L_3}\right) &= \left(\frac{U_2}{L_2}\right)^{\frac{1}{2}} = \left(\frac{U_1}{L_1}\right)^{\left(\frac{1}{2}\right)^2}, \end{aligned}$$

$$\begin{aligned} & \vdots \\ \left(\frac{U_k}{L_k}\right) &= \left(\frac{U_{k-1}}{L_{k-1}}\right)^{\frac{1}{2}} = \dots = \left(\frac{U_1}{L_1}\right)^{\left(\frac{1}{2}\right)^{k-1}}. \end{aligned}$$

Because $\left(\frac{U_k}{L_k}\right) > (1 + \varepsilon)$ and $\frac{U_1}{L_1} \leq \left(\frac{|V|}{1+\rho}\right) + 1$, we have $k \leq \log_{\frac{1}{2}} \log_{\frac{|V|}{1+\rho}} (1 + \varepsilon) + 1$. Hence, k is $O(\log \log |V| - \log \log (1 + \varepsilon))$. Then step 3.2.2 can be done in $O(|V|^{o(\frac{1}{\rho})} * (|E| \log^2 |V| + |V| \log^3 |V|))$ time by Ravi's algorithm [18]. Hence, Algorithm PTAS-BMST is a polynomial time approximation scheme for the BMST problem. $\square$

Now, we improve the approximation ratio to $(1, 1 + \varepsilon)$ for the BMST problem. This algorithm is the modification of the one designed in [8] for the restricted shortest path problem.

Algorithm Modify-PTAS-BMST:

Input: A graph $G = (V, E)$ with a weight function and a cost function on edges, an integer bound B , and two real numbers $\varepsilon > 0$ and $0 < \gamma < 1$.

Output: A spanning tree T_{APX} of G subject to $W(T_{APX}) \leq B$.

1. **For** each edge in E , swap cost to weight and weight to cost, respectively. **end for**
2. Let an initial upper bound $UB = |V| * c_J$ and an initial lower bound $LB = c_J$.
3. **If** $UB \leq (1 + \varepsilon)LB$ **then** let $T_{APX} = T_J$ and return.

Else

3.1. Let $L = LB$ and $U = UB$.

3.2. Repeat the following steps until the condition (1) $U \leq (1 + \varepsilon)L$ or (2) $U \leq L + 1$ holds.

3.2.1. Let a real number $\rho = \left(\frac{U}{L}\right)^\gamma - 1$, and a middle value $M = \sqrt{\frac{U * L}{1 + \rho}}$.

3.2.2. Use algorithm $\mathcal{A}_R(\rho, M)$ to find a spanning tree T .

3.2.3. /* if $W(T) > B$ then we have a lower bound M else we have an upper bound $(1 + \rho)M$ and $C(T)$ is less than or equal to this upper bound.*/
If $W(T) \leq B$ then let $U = (1 + \rho)M$ and $T_{APX} = T$ else let $L = M$.

The approximation ratio of Algorithm Modify-PTAS-BMST is shown in the next lemma.

Lemma 4: The spanning tree T_{APX} is a $(1, 1 + \varepsilon)$ -approximation solution for the BMST problem.

Proof: Let the upper bound U_i , the middle value M_i , the lower bound L_i , and the real number ρ_i be parameters at the beginning of the i -th iteration of step 3.2 for Algorithm Modify-PTAS-BMST. Let T_i denote a spanning tree after running the i -th iteration of step 3.2.2. If $W(T_i) \leq B$, we have $U_{i+1} = (1 + \rho_i)M_i = (U_i^{1+\gamma} * L_i^{1-\gamma})^{\frac{1}{2}}$ and $L_{i+1} = L_i$. Otherwise, if $W(T_i) > B$, we have $L_{i+1} = M_i = (U_i^{1-\gamma} * L_i^{1+\gamma})^{\frac{1}{2}}$ and $U_{i+1} = U_i$. Hence, $\frac{U_{i+1}}{L_{i+1}} = \left(\frac{U_i}{L_i}\right)^{\frac{\gamma+1}{2}}$. Note that it successively narrows down the range between the lower bound L and the upper bound U after each iteration of step 3.2, since $0 < \gamma < 1$. Let k be the number of iterations of step 3.2. After performing this algorithm, we have either $U_{k+1} \leq (1 + \varepsilon)L_{k+1}$ or $U_{k+1} \leq L_{k+1} + 1$. For the former case, we have

$$\begin{aligned} C(T_{APX}) &\leq U_{k+1} \leq (1 + \varepsilon)L_{k+1} \\ &\leq (1 + \varepsilon)C(T_{OPT}). \end{aligned}$$

For the latter case, we have

$$C(T_{APX}) \leq U_{k+1} \leq (C(T_{OPT}) + 1).$$

$\square$

Therefore, we have the following theorem.

Theorem 5: The BMST problem admits a PTAS. For any constant $\varepsilon > 0$, a $(1, 1 + \varepsilon)$ -approximation algorithm for the BMST problem can be found in polynomial time.

Proof: Clearly, Algorithm Modify-PTAS-BMST returns a $(1, 1 + \varepsilon)$ -approximation solution for the BMST problem by Lemma 4. Next, we analyze the time complexity of Algorithm Modify-PTAS-BMST as follows. Let k be the number of iterations of step 3.2. We also let L_1 and U_1 be the initial lower bound and upper bound, respectively. Then we have

$$\begin{aligned} & \frac{U_1}{L_1}, \\ \left(\frac{U_2}{L_2}\right) &= \left(\frac{U_1}{L_1}\right)^{\frac{\gamma+1}{2}}, \\ \left(\frac{U_3}{L_3}\right) &= \left(\frac{U_2}{L_2}\right)^{\frac{\gamma+1}{2}} = \left(\frac{U_1}{L_1}\right)^{\left(\frac{\gamma+1}{2}\right)^2}, \\ & \vdots \\ \left(\frac{U_k}{L_k}\right) &= \left(\frac{U_{k-1}}{L_{k-1}}\right)^{\frac{\gamma+1}{2}} = \dots = \left(\frac{U_1}{L_1}\right)^{\left(\frac{\gamma+1}{2}\right)^{k-1}}, \end{aligned}$$

since $\frac{U_{i+1}}{L_{i+1}} = \left(\frac{U_i}{L_i}\right)^{\frac{\gamma+1}{2}}$.

Initially, $\frac{U_1}{L_1} = |V|$. After running Algorithm Modify-PTAS-BMST, we have $(\frac{U_{k+1}}{L_{k+1}}) \leq (1 + \varepsilon)$. Step 3.2.2 can be done in $O(|V|^{o(\frac{1}{\rho})} * (|E| \log^2 |V| + |V| \log^3 |V|))$ time by Ravi's algorithm [18]. Let I be the smallest index j such that $|V|^{o(\frac{1}{\rho_j})} > |V|$, for $1 \leq j \leq k$. For all $1 \leq j \leq I - 1$, the time is $\sum_{j=1}^{I-1} O(|V|^{o(\frac{1}{\rho_j})} (|E| \log^2 |V| + |V| \log^3 |V|))$. It is clear $(\frac{U_I}{L_I}) < 2^{\frac{1}{\gamma}}$ with $(\frac{U_{I-1}}{L_{I-1}}) \geq 2^{\frac{1}{\gamma}}$. Hence we have $\log_{\frac{\gamma+1}{2}}(\frac{1}{\gamma * \log_2 |V|}) + 1 \geq I - 1 > \log_{\frac{\gamma+1}{2}}(\frac{1}{\gamma * \log_2 |V|})$ since $(\frac{U_I}{L_I}) = (\frac{U_1}{L_1})^{((\frac{\gamma+1}{2})^{I-1})}$. If $I - 1 \geq 1$, the complexity is at most $O(|V| (1 - \log_{\frac{\gamma+1}{2}} \gamma - \log_{\frac{\gamma+1}{2}}(\log_2 |V|)) (|E| \log^2 |V| + |V| \log^3 |V|))$, for all $1 \leq j \leq I - 1$. Then for all $I \leq j \leq k$, the time complexity is $\sum_{j=I}^k O(|V|^{o(\frac{1}{\rho_j})} (|E| \log^2 |V| + |V| \log^3 |V|))$. Since $(\frac{U_k}{L_k}) > (1 + \varepsilon)$ with $(\frac{U_k}{L_k}) = (\frac{U_1}{L_1})^{((\frac{\gamma+1}{2})^{(k-1)})}$, we have $k < \log_{\frac{\gamma+1}{2}}(\frac{\log(1+\varepsilon)}{\log |V|}) + 1$. Moreover, $(\frac{1}{\rho_I}) \leq (\frac{1}{\rho_{I+1}}) \leq \dots \leq (\frac{1}{\rho_k})$. Hence we have the time complexity at most $O(|V|^{o(\frac{1}{\rho_k})} (1 + \log_{\frac{\gamma+1}{2}} \log(1 + \varepsilon) + \log_{\frac{\gamma+1}{2}} \gamma) (|E| \log^2 |V| + |V| \log^3 |V|))$ for all $I \leq j \leq k$. Because γ is a constant, the total time complexity is at most $O((|V| \log \log |V| + |V|^{\frac{1}{(1+\varepsilon)^{\gamma-1}}} \log \log(1 + \varepsilon)) (|E| \log^2 |V| + |V| \log^3 |V|))$, where $\frac{1}{\rho_k} \leq \frac{1}{(1+\varepsilon)^{\gamma-1}}$, $0 < \gamma < 1$. $\square$

Let $\varepsilon = 1$ and $\gamma \approx 1$. Use Algorithm Modify-PTAS-BMST to find a feasible solution for the BMST problem with ratio (1,2). The total time complexity of this algorithm is at most $O((|V|^{\frac{1}{2^{\gamma-1}}} + |V| \log \log |V|) (|E| \log^2 |V| + |V| \log^3 |V|))$. Then we can apply Hassin and Levin's [11] algorithm (i.e. Lagrangian relaxation and matroid intersection), the approximation ratio can be improved to $(1, 1 + 4\varepsilon)$ with the same time complexity of Hassin and Levin's algorithm (i.e. $O((\frac{1}{\varepsilon^2})^{(\frac{1}{\varepsilon})} |V|^3)$).

3 Conclusion

In this paper, we presented two polynomial time approximation schemes for the BMST problem. Using the second PTAS with Lagrangian relaxation and matroid intersection, the approximation ratio of Hassin and Levin's algorithm [11] can be improved to the ratio of $(1, 1 + 4\varepsilon)$. A open problem left in the paper could involve studying whether there exists a fully polynomial time approximation scheme for the BMST problem.

References

- [1] V. Aggarwal, Y. Aneja, K. Nair, Minimal spanning tree subject to a side constraint, *Computers and Operations Research* 9 (1982) pp. 287–296.
- [2] P. Chalasani, R. Motwani, Approximating capacitated routing and delivery problems, *SIAM Journal on Computing* 28 (1999) pp. 2133–2149.
- [3] G.T. Chen, G.L. Xue, K-pair delay constrained minimum cost routing in undirected networks, in: *Proc. of the 12th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, Washington, D.C., USA, 2001, pp. 230–231.
- [4] G.T. Chen, G.L. Xue, A PTAS for weight constrained Steiner trees in series-parallel graphs, *Theoretical Computer Science* 304 (2003) pp. 237–247.
- [5] C.H. Chow, On multicast path finding algorithms, in: *Proc. of IEEE INFOCOM*, Bal Harbour, FL, USA, 1991, pp. 1274–1283.
- [6] W.J. Cook, W.H. Cunningham, W.R. Pulleyblank, A. Schrijver, *Combinatorial Optimization*, John Wiley and Sons, New York, 1998.
- [7] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, *Introduction to Algorithm*, third ed., MIT Press, Cambridge, 2001.
- [8] F. Ergun, R. Sinha, L. Zhang, An improved FPTAS for restricted shortest path, *Information Processing Letters* 83 (2002) pp. 287–291.
- [9] A. Goel, K.G. Ramakrishnan, D. Kataria, D. Logothetis, Efficient computation of delay-sensitive routes from one source to all destinations, in: *Proc. of IEEE INFOCOM*, Anchorage, AK, USA, 2001, pp. 854–858.
- [10] R. Hassin, Approximation schemes for the restricted shortest path problems, *Mathematics of Operations Research* 17 (1992) pp. 36–42.
- [11] R. Hassin, A. Levin, An efficient polynomial time approximation scheme for the constrained minimum spanning tree problem using matroid intersection, *SIAM Journal on Computing* 33 (2004) pp. 261–268.
- [12] S.P. Hong, S.J. Chung, B.H. Park, A fully polynomial bicriteria approximation scheme

- for the constrained spanning tree problem, *Operations Research Letters* 32 (2004), pp. 233–239.
- [13] B. Kadaba and J. Jaffe, Routing to multiple destinations in computer networks, *IEEE Transactions on Communications* 31 (1983) pp. 343–351.
- [14] V.P. Kompella, J.C. Pasquale, G.C. Polyzos, Multicasting for multimedia applications, *IEEE/ACM Transactions on Networking* (1993) pp. 286–292.
- [15] E. Lawler, *Combinatorial Optimization: Networks and Matroids*, Holt, Rinehart and Winston, New York, 1976.
- [16] D. Lorenz, D. Raz, Simple efficient approximation scheme for the restricted shortest path problem, *Operations Research Letters* 28 (2001) pp. 213–219.
- [17] M.V. Marathe, R. Ravi, R. Sundaram, S.S. Ravi, D.J. Rosenkrantz, H.B. Hunt III, Bicriteria network design problems, *Journal of Algorithms* 28 (1998) pp. 142–171.
- [18] R. Ravi, M. Goemans, The constrained minimum spanning tree problem, *Lecture Notes in Computer Science* 1097 (1996) pp. 66–75.
- [19] Z. Wang, On the complexity of quality of service routing, *Information Processing Letters* 69 (1999) pp. 111–114.
- [20] G. Xue, W. Zhang, J. Tang, K. Thulasiraman, Polynomial time approximation algorithms for multi-constrained QoS routing, *IEEE/ACM Transactions on Networking* 16 (2008) pp. 656–669.